EX02 - Ecuacion ideal del cohete - Imports y logaritmos

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex02_rocket_equation.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Calcular Delta_v a partir de Isp, g0, masa inicial m0 y masa final mf.

Variables / elementos del problema

Nombre	Significado
Isp	impulso especifico [s]
g0	gravedad de referencia [m/s^2]
m0	masa inicial [kg]
mf	masa final [kg]
Delta_v	incremento ideal de velocidad [m/s]

Ecuacion o logica en papel

```
Delta_v = Isp * g0 * ln(m0 / mf)
```

Comprobacion manual

Caso de control: Isp=250, g0=9.81, m0=500000, mf=250000. La relacion de masas es 2 y ln(2)~0.6931, por lo que Delta_v~1699.7 m/s. El archivo actual usa mf=500000 y por eso entra en la validacion m0<=mf.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- variables
- if / elif / else
- print()

Nuevo en este ejercicio:

- import numpy as np
- llamada a np.log()
- variable intermedia mass_ratio
- varias validaciones encadenadas

3. Conceptos Python nuevos

1. Importar una libreria

Sintaxis: `import numpy as np`

import carga un modulo. as np crea un alias corto para poder escribir np.log().

2. Llamar una funcion

Sintaxis: `np.log(mass_ratio)`

Los parentesis significan que ejecutamos log y le entregamos mass_ratio como argumento. np.log devuelve el logaritmo natural.

3. Cadena de decisiones

Sintaxis: `if ... elif ... else:`

Python prueba las ramas de arriba abajo. Cuando encuentra la primera condicion True, ejecuta esa rama y salta las demas.

4. Codigo paso a paso

Paso 1. Importar NumPy

```
import numpy as np
```

Necesitamos una funcion matematica que no forma parte de la sintaxis basica: np.log().

Paso 2. Definir entradas

```
Isp = 250
g0 = 9.81
m0 = 500000
mf = 500000
```

Cada variable representa una magnitud del modelo.

Paso 3. Validar antes del calculo

```
if Isp < 0:
    ...
elif m0 <= 0:
    ...
elif mf <= 0:
    ...
elif m0 <= mf:
    ...
```

<= significa menor o igual. El ejercicio impide llegar al calculo cuando una entrada no cumple las reglas definidas.

Paso 4. Calcular la relacion de masas

```
mass_ratio = m0 / mf
```

Una variable intermedia hace visible una parte importante de la ecuacion y facilita depurarla.

Paso 5. Aplicar el logaritmo

```
delta_v = Isp * g0 * np.log(mass_ratio)
```

Esta linea es la traduccion directa de la ecuacion matematica.

5. Flujo de ejecucion

```
inputs
   -> validation chain
   -> mass_ratio
   -> np.log(mass_ratio)
   -> delta_v
   -> print
```

6. Resultado / comprobacion esperada

Con el archivo tal cual, m0==mf y se imprime el mensaje de validacion. Para probar el calculo cambia mf a 250000 y espera ~1699.7 m/s.

7. Errores tipicos al empezar

- Confundir np.log con log base 10.
- Poner el calculo antes de validar.
- No entender que solo se ejecuta la primera rama True.
- Olvidar que / produce una division real.

8. Si solo recuerdas cinco cosas

- import carga herramientas externas.
- np es un alias.
- funcion(...) ejecuta una funcion.
- elif permite ordenar validaciones.
- Las variables intermedias hacen el codigo mas legible.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.